Pet Heaven – Demo Video Script

ISIT207 Assignment 3, Part B · Narrated screen walk-through · Target length: 6 to 7 minutes

How to use this script: read the **Say** lines in your own words, and do the **On screen** actions while you speak. Rehearse once or twice before recording. Replace [Your name] and anything in brackets.

Scene 1: Introduction (0:00 – 0:25)

On screen: Live site home page: https://petheavenassignment3.netlify.app/#/

Say: Hello, my name is [Your name], and this is my ISIT207 Assignment 3 demonstration. I built a single page web application for Pet Heaven, a charity that cares for abandoned cats and dogs and lets visitors request to adopt or release a pet. It is built with React, and this is the live deployed version on Netlify.

Scene 2: Design ideas (0:25 – 1:15)

On screen: Stay on the home page. Slowly scroll from the hero to the footer.

Say: My design aim was a warm, calm and trustworthy look that suits a charity. The colour palette uses a deep forest green, soft sage green, a terracotta accent and a warm orange for highlights. For typography I paired Playfair Display for headings with DM Sans for body text, so the page feels friendly but still readable.

The layout has a clear visual hierarchy: a large headline and call to action buttons at the top, the charity's key numbers, a preview of pets, a story from the community, and a call to action to join. The header navigation and the footer, which holds the contact details, appear on every page so the user is never lost.

Scene 3: About us and Our mission (1:15 – 1:45)

On screen: Click **About us**, point at the three facility photos. Then click **Our mission**.

Say: The About us page explains who we are and shows the facilities: the shelter, the care room and the play yard. The Our mission page states the purpose of the society and its key figures. Together they cover the requirement to communicate the purpose, facilities and information of the society.

Scene 4: Pet directory (1:45 – 2:50)

On screen: Click **Adopt**. Click the **Dog** tab, then **Cat**, then **All pets**. Type "calm" in the search box, then clear it. Click one pet card to open the popup. Press Escape to close it.

Say: The Adopt page is the pet directory, with sixteen pets, eight dogs and eight cats. Each card shows a photo, name, breed, age and personality tags. I added tabs to filter by dogs or cats, and a search box that matches names, breeds and traits. For example, searching "calm" shows only the calmer pets. Clicking a card opens a detail popup with the pet's description, and it can be closed with the close button or the Escape key. Each card is the same reusable PetCard component, fed from one data file.

Scene 5: Pet quiz (2:50 – 3:20)

On screen: Click **Take the pet quiz**. Answer the three questions and show the recommended pet.

Say: As an extra feature, I added a short pet quiz. It asks about living space, activity level and time at home, scores every pet against the answers, and recommends the best match.

Scene 6: Member accounts (3:20 – 4:40)

On screen: Click **Member login**. Show the demo account box. Try an unknown email, then the demo email with a wrong password. Then click **New here? Create a member account** and type a weak password slowly to show the strength bar. Finish creating a real account.

Say: Visitors can register as a member. On the login screen, an email that has no account shows a clear error, and a wrong password shows a different error. There is also a demo account for anyone testing the site.

On the sign up screen, the user must enter their name and a password of at least eight characters that mixes letters and numbers. As I type, the strength bar and checklist update, from too weak up to strong. Once the account is created, I am logged in and my name appears in the header.

To keep the project front end only, the accounts are simulated in the browser's local storage, with passwords stored as hashes rather than plain text. A real system would use a server and database.

Scene 7: Adoption and release forms (4:40 – 5:40)

On screen: Open a pet card and click **Adopt**. Submit the form empty to show errors, enter a bad phone number, then fill it in correctly and submit. Then open **I need to release a pet** on the home page and do the same quickly.

Say: Adopting a pet, releasing a pet and sharing a story all require an account. When I am logged out, these forms show a members only message. Now that I am logged in, the adoption form already has my name and email filled in, and the pet I chose is selected. If I submit it empty, each field shows an error, and a badly formatted phone number is rejected. When everything is valid, the form shows a confirmation that the request has been sent to the administrator. The release form works the same way. Both forms are built from one reusable FormCard component that takes a list of fields, which keeps the code short and consistent.

Scene 8: Stories, care tips and contact (5:40 – 6:30)

On screen: Click **Stories** and post a short story. Click **Care tips** and play a few seconds of the video. Click **Contact**, show the details and the map.

Say: On the Stories page members can post their own story, and it appears straight away in the list. The Care tips page has practical advice for settling in a new pet, plus an embedded video from MiamiDadeCounty. The Contact page shows the address, phone number, email and opening hours, with a Google map, and a message form with validation.

Scene 9: Extras and responsiveness (6:30 – 7:00)

On screen: Click the moon icon to switch to dark mode, then back. Press F12, click the device toolbar and choose a phone size to show the layout.

Say: There is a light and dark theme toggle. Page changes fade in, popups animate open and buttons react on hover. The layout is responsive, so on a phone the navigation wraps and the pet cards stack in one column.

Scene 10: How it is built and closing (7:00 – 7:30)

On screen: Switch to VS Code and show the **src** folder tree, with the components and pages folders.

Say: Behind the scenes, this is a React single page application using React Router, so changing pages never reloads the browser. The code is split into reusable components such as the navigation bar, pet card, pet popup, form card, password meter and footer. State is handled with useState, and the logged in user and theme are shared through React context. All pet data comes from one data file, so adding a pet needs only one new entry and one photo.

The site is deployed on Netlify, and the link is in my report. Thank you for watching.

Recording checklist

- Record the live Netlify site in Chrome, not localhost, and use the demo account or a fresh sign up.
- Close other tabs and apps, turn off notifications, and keep the browser zoom at 100 percent.
- Do a short test recording first to check that your voice is clear and the audio is recorded.
- Speak slowly, and pause for a second after each click so the viewer can follow.
- Upload to Google Drive, set sharing to "Anyone with the link can view", and paste that link into your report.